

Assignment 1 - Implement multi-threaded client/server Process communication using RMI.

AddServerIntf.java

```
import java.rmi.*;

public interface AddServerIntf extends Remote {
    double add(double d1, double d2) throws RemoteException;
}
```

AddServerImpl.java

```
import java.rmi.*;
import java.rmi.server.*;

public class AddServerImpl extends UnicastRemoteObject
implements AddServerIntf {
    public AddServerImpl() throws RemoteException {}

    public double add(double d1, double d2) throws RemoteException {
        return d1 + d2;
    }
}
```

AddServer.java

```
import java.rmi.*;

public class AddServer {
    public static void main(String args[]) {
        try {
            AddServerImpl addServerImpl = new AddServerImpl();
            Naming.rebind("AddServer", addServerImpl);
        }
    }
}
```

```
        System.out.println("Server is running...");
    } catch (Exception e) {
        System.out.println("Exception: " + e);
    }
}
}
```

AddClient.java

```
import java.rmi.*;
```

```
public class AddClient {
    public static void main(String args[]) {
        try {
            String addServerURL = "rmi://" + args[0] + "/AddServer";
            AddServerIntf addServerIntf = (AddServerIntf)
Naming.lookup(addServerURL);

            System.out.println("The first number is: " + args[1]);
            double d1 = Double.parseDouble(args[1]);

            System.out.println("The second number is: " + args[2]);
            double d2 = Double.parseDouble(args[2]);

            System.out.println("The sum is: " + addServerIntf.add(d1, d2));
        } catch (Exception e) {
            System.out.println("Exception: " + e);
        }
    }
}
```

Output:

Terminal 1st-

```
javac *.java
start rmiregistry
java AddServer
javac --release 11 *.java
java -version
javac -version
java AddServer
Server is running...
```

Terminal 2nd-

```
java AddClient localhost 8 9
The first number is: 8
The second number is: 9
The sum is: 17.0
```

```
java AddClient localhost 5 6
The first number is: 5
The second number is: 6
The sum is: 11.0
```

Assignment 4 Implement Berkeley algorithm for clock synchronization

BerkeleyAlgorithm.java

```
import java.util.*;

public class BerkeleyAlgorithm {
    public static void main(String[] args) {
        // Step 1: Create nodes with different clock times
        List<ClockNode> nodes = new ArrayList<>();
        nodes.add(new ClockNode(0, 100, true)); // Master node
```

```
nodes.add(new ClockNode(1, 98, false));
nodes.add(new ClockNode(2, 105, false));
nodes.add(new ClockNode(3, 102, false));
nodes.add(new ClockNode(4, 95, false));
```

```
System.out.println("Initial Times:");
for (ClockNode node : nodes) {
    System.out.println(node);
}
```

```
// Step 2: Master collects time differences
ClockNode master = nodes.get(0);
int masterTime = master.time;
int totalDiff = 0;
int count = 0;
```

```
System.out.println("\nCalculating time differences:");
for (ClockNode node : nodes) {
    if (!node.isMaster) {
        int diff = node.time - masterTime;
        System.out.println("Node " + node.id + " reports difference: "
+ diff + "s");
        totalDiff += diff;
        count++;
    }
}
```

```
// Step 3: Calculate average adjustment (master doesn't count
itself)
int averageDiff = totalDiff / (count + 1); // Include master for fair
average
```

```

        System.out.println("\nAverage time difference: " + averageDiff +
"s");

        // Step 4: Master sends time adjustments to all
        for (ClockNode node : nodes) {
            int adjustment = averageDiff - (node.time - masterTime);
            node.adjustTime(adjustment);
            System.out.println("Node " + node.id + " adjusted by: " +
adjustment + "s");
        }

        // Final times
        System.out.println("\nSynchronized Times:");
        for (ClockNode node : nodes) {
            System.out.println(node);
        }
    }
}

```

ClockNode.java

```

public class ClockNode {
    int id;
    int time; // in seconds
    boolean isMaster;

    public ClockNode(int id, int time, boolean isMaster) {
        this.id = id;
        this.time = time;
        this.isMaster = isMaster;
    }
}

```

```

    public void adjustTime(int offset) {
        time += offset;
    }

    @Override
    public String toString() {
        return "Node " + id + " time: " + time;
    }
}

```

Output:

```

javac ClockNode.java BerkeleyAlgorithm.java
zip\Assi 4> java BerkeleyAlgorithm
Initial Times: Node 0 time: 100 Node 1 time: 98 Node 2 time: 105
Node 3 time: 102 Node 4 time: 95 Calculating time differences: Node
1 reports difference: -2s Node 2 reports difference: 5s Node 3 reports
difference: 2s Node 4 reports difference: -5s Average time difference:
0s Node 0 adjusted by: 0s Node 1 adjusted by: 2s Node 2 adjusted
by: -5s Node 3 adjusted by: -2s Node 4 adjusted by: 5s Synchronized
Times: Node 0 time: 100 Node 1 time: 100 Node 2 time: 100 Node 3
time: 100 Node 4 time: 100

```

Assignment 5

Implement token ring based mutual exclusion algorithm.

TokenRingMain.java

```

public class TokenRingMain {
    public static void main(String[] args) {
        // Create a TokenRing with 5 processes
        TokenRing tokenRing = new TokenRing(5);
    }
}

```

```
        tokenRing.startRing();
    }
}
```

Process.java

```
class Process implements Runnable {
    private int id;
    private TokenRing tokenRing;

    public Process(int id, TokenRing tokenRing) {
        this.id = id;
        this.tokenRing = tokenRing; // Assign the tokenRing instance
    }

    public int getId() {
        return id;
    }

    @Override
    public void run() {
        while (true) {
            try {
                // Wait for the token
                if (id == tokenRing.getCurrentProcessWithToken().getId()) {
                    enterCriticalSection();
                    passTokenToNextProcess();
                }
                Thread.sleep(500); // Simulate waiting time
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

```

    }
}

public void enterCriticalSection() {
    System.out.println("Process " + id + " entered critical section.");
    try {
        Thread.sleep(2000); // Simulate work in critical section
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    System.out.println("Process " + id + " left critical section.");
}

public void passTokenToNextProcess() {
    int nextIndex = tokenRing.getNextProcessIndex(id);
    tokenRing.passToken(id, nextIndex);
}
}

```

TokenRing.java

```

import java.util.*;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

class TokenRing {
    private List<Process> processes;
    private int currentTokenHolderIndex;
    private final Lock lock = new ReentrantLock();

    public TokenRing(int numProcesses) {
        processes = new ArrayList<>();
    }
}

```

```
        currentTokenHolderIndex = 0; // First process holds the token  
initially  
        for (int i = 0; i < numProcesses; i++) {  
            // Pass the tokenRing instance to each Process when created  
            processes.add(new Process(i, this));  
        }  
    }
```

```
    public void startRing() {  
        // Start the processes  
        for (Process process : processes) {  
            new Thread(process).start();  
        }  
    }
```

```
    public void passToken(int fromIndex, int toIndex) {  
        lock.lock();  
        try {  
            System.out.println("Process " +  
processes.get(fromIndex).getId() + " passes the token to Process " +  
processes.get(toIndex).getId());  
            currentTokenHolderIndex = toIndex;  
        } finally {  
            lock.unlock();  
        }  
    }
```

```
    public Process getCurrentProcessWithToken() {  
        return processes.get(currentTokenHolderIndex);  
    }
```

```
    public int getNextProcessIndex(int currentIndex) {
```

```
        return (currentIndex + 1) % processes.size();  
    }  
}
```

Output:

```
PS E:\DS PR zip\TokenRing> javac TokenRing.java  
TokenRingMain.java  
>>  
PS E:\DS PR zip\TokenRing> java TokenRingMain  
>>  
Process 0 entered critical section.  
Process 0 left critical section.  
Process 0 passes the token to Process 1  
Process 1 entered critical section.  
Process 1 left critical section.  
Process 1 passes the token to Process 2  
Process 2 entered critical section.  
Process 2 left critical section.  
Process 2 passes the token to Process 3  
Process 3 entered critical section.  
Process 3 left critical section.  
Process 3 passes the token to Process 4  
Process 4 entered critical section.  
Process 4 left critical section.  
Process 4 passes the token to Process 0  
Process 0 entered critical section.  
Process 0 left critical section.  
Process 0 passes the token to Process 1  
Process 1 entered critical section.  
Process 1 left critical section.  
Process 1 passes the token to Process 2
```

Process 2 entered critical section.
Process 2 left critical section.
Process 2 passes the token to Process 3
Process 3 entered critical section.
PS E:\DS PR zip\TokenRing>

Assignment 6

Implement Bully and Ring algorithm for leader election.

BullyAlgorithm.java

```
import java.util.*;

public class BullyAlgorithm {
    private List<Process> processes;
    private int leader = -1;

    public BullyAlgorithm(int numProcesses) {
        processes = new ArrayList<>();
        for (int i = 0; i < numProcesses; i++) {
            processes.add(new Process(i));
        }
    }

    public void startElection(int initiatorId) {
        Process initiator = processes.get(initiatorId);
        System.out.println("Process " + initiatorId + " is initiating the election.");
        initiator.initiateElection();
    }
}
```

```

public class Process {
    private int id;
    private boolean isAlive;
    private boolean isLeader;

    public Process(int id) {
        this.id = id;
        this.isAlive = true;
        this.isLeader = false;
    }

    public int getId() {
        return id;
    }

    public void initiateElection() {
        for (int i = id + 1; i < processes.size(); i++) {
            if (processes.get(i).isAlive) {
                System.out.println("Process " + id + " sends election
message to " + i);
            }
        }

        boolean responseReceived = false;
        for (int i = id + 1; i < processes.size(); i++) {
            if (processes.get(i).isAlive) {
                responseReceived = true;
                break;
            }
        }
    }
}

```

```

        if (!responseReceived) {
            leader = id;
            isLeader = true;
            System.out.println("Process " + id + " becomes the leader.");
        } else {
            System.out.println("Election aborted, higher ID process
wins.");
        }
    }

    public void fail() {
        isAlive = false;
        System.out.println("Process " + id + " has failed.");
    }
}

public static void main(String[] args) {
    BullyAlgorithm bully = new BullyAlgorithm(5);

    // Simulate a process initiating an election
    bully.startElection(0);
}
}

```

RingAlgorithm.java

```

import java.util.*;

public class RingAlgorithm {
    private List<Process> processes;
    private int leader = -1;

    public RingAlgorithm(int numProcesses) {

```

```
processes = new ArrayList<>();  
for (int i = 0; i < numProcesses; i++) {  
    processes.add(new Process(i));  
}  
}
```

```
public void startElection() {  
    System.out.println("Election started.");  
    Process tokenHolder = processes.get(0);  
    tokenHolder.sendToken();  
}
```

```
public class Process {  
    private int id;  
    private boolean isAlive;  
    private boolean isLeader;
```

```
    public Process(int id) {  
        this.id = id;  
        this.isAlive = true;  
        this.isLeader = false;  
    }
```

```
    public int getId() {  
        return id;  
    }
```

```
    public void sendToken() {  
        System.out.println("Process " + id + " sends the token.");  
        int nextProcessIndex = (id + 1) % processes.size();  
        Process nextProcess = processes.get(nextProcessIndex);  
        nextProcess.receiveToken(this);  
    }
```

```

    }

    public void receiveToken(Process previousProcess) {
        System.out.println("Process " + id + " receives the token from
Process " + previousProcess.getId());
        if (id > previousProcess.getId()) {
            sendToken();
        } else {
            leader = id;
            isLeader = true;
            System.out.println("Process " + id + " is elected as the
leader.");
        }
    }

    public void fail() {
        isAlive = false;
        System.out.println("Process " + id + " has failed.");
    }
}

public static void main(String[] args) {
    RingAlgorithm ring = new RingAlgorithm(5);

    // Start the election by sending the token
    ring.startElection();
}
}

```

Output:

```

PS E:\DS PR zip\BullyAlgo> javac BullyAlgorithm.java
RingAlgorithm.java

```

>>

PS E:\DS PR zip\BullyAlgo> java BullyAlgorithm

>>

Process 0 is initiating the election.

Process 0 sends election message to 1

Process 0 sends election message to 2

Process 0 sends election message to 3

Process 0 sends election message to 4

Election aborted, higher ID process wins.

PS E:\DS PR zip\BullyAlgo> java RingAlgorithm

>>

Election started.

Process 0 sends the token.

Process 1 receives the token from Process 0

Process 1 sends the token.

Process 2 receives the token from Process 1

Process 2 sends the token.

Process 3 receives the token from Process 2

Process 3 sends the token.

Process 4 receives the token from Process 3

Process 4 sends the token.

Process 0 receives the token from Process 4

Process 0 is elected as the leader.

PS E:\DS PR zip\BullyAlgo>

Assignment 7

Create a simple web service and write any distributed application to consume the web service